← Índice de la Bible

**estado**: accepted · **fecha**: 2026-07-17 · **fecha-resolucion**: 2026-07-18

# ADR-0002 — Plataforma de generación del sitio documental de la Bible

## Contexto

El acta D8 (rango 1) aprobó **Git + Markdown** como plataforma documental y ordenó que la documentación *"deberá poder convertirse automáticamente en sitio web con MkDocs Material o Docusaurus (elección pendiente — se resolverá vía ADR con las 4 opciones de nivel)"*. El acta D12 añadió que, al aprobarse un documento, *"se generan automáticamente su versión Markdown, PDF y HTML navegable, y se actualiza el índice de la Bible"*, e impactó explícitamente `herramientas/publicar.py`.

Hoy existe un publicador propio ya operativo ( `herramientas/publicar.py` ) que satisface el mínimo del D12 (Markdown + PDF + HTML), pero **no es** ninguno de los dos generadores que el CEO nombró en el D8 y carece de búsqueda, de navegación jerárquica y de render de diagramas. Este ADR resuelve, en los cuatro niveles del acta D3, con qué plataforma se genera el sitio definitivo de la Bible. La decisión de fondo (qué nivel) **corresponde al CEO** (D3: *"el CEO decidirá cuál implementar"*); este ADR presenta las alternativas reales y la recomendación argumentada del proponente.

*Formato de alternativas:* **aplican** los 4 niveles del acta D3 (Económica · Balanceada · Profesional · Enterprise), porque la decisión llega al CEO con opciones de implementación reales.

## Alternativas consideradas

### A. Económica — mantener el publicador actual `herramientas/publicar.py`

Descripción (tras leer el script): utilidad Python 3 (mandato D12) que ya opera. Recorre `decisiones/*.md` , `adr/*.md` , `docs/*/*.md` , `INDICE.md` , `README.md` y `CHANGELOG.md` ; publica solo los documentos `Aprobado` / `Vinculante` / `accepted` (excluye la plantilla de ADR). Convierte Markdown a HTML con `python3-markdown` (extensiones `tables` , `fenced_code` , `toc` ), reescribe los enlaces `.md` → `.html` , incrusta una hoja CSS serif, genera un `index.html` navegable agrupado por carpeta y produce el PDF de

cada documento vía Chrome headless ( `--print-to-pdf` ). Invocación manual: `python3 herramientas/publicar.py [--sin-pdf]` .

Pros: costo cero; **ya operativo**; sin dependencias nuevas más allá de `python3-markdown` y Chrome; 100% local y estático (refuerza el ADR-0001); control total del pipeline; cubre el mínimo del D12 (MD + PDF + HTML + índice). Contras: **no es** lo que el D8 nombró para el sitio (*"MkDocs Material o Docusaurus"*); sin búsqueda; navegación plana (índice por carpeta, sin sidebar jerárquico); **no renderiza Mermaid** (los diagramas quedan como bloques de código); sin versionado de sitio; estilo básico; mantenimiento manual del script.

## B. Balanceada — MkDocs Material

Generador de sitios estáticos en Python. Navegación jerárquica automática, búsqueda cliente integrada, render de Mermaid (vía `pymdownx.superfences` /plugin), tema accesible con modo claro/oscuro, permalinks. Configuración declarativa ( `mkdocs.yml` ); build/serve local y estático. Versionado de sitio disponible con `mike` si se requiere.

Pros: es **uno de los dos generadores que el CEO nombró** (D8); permanece en el stack Python ya presente ( `python3-markdown` ), con curva de adopción mínima; búsqueda + navegación + Mermaid **de fábrica**; 100% local y estático (compatible con ADR-0001, sin servicios externos); bajo mantenimiento, **sostenible por el equipo real** (ADR-0004 regla 13). Contras: introduce la dependencia MkDocs + plugins; el PDF deja de ser nativo (se conserva `publicar.py --sin-pdf` para PDF, o se añade un plugin); una migración inicial de layout.

## C. Profesional — Docusaurus

Generador React/Node. MDX (componentes dentro de la documentación), **versionado nativo de documentación**, i18n, tema personalizable, búsqueda local o Algolia.

Pros: es **el otro generador que el CEO nombró** (D8); versionado nativo de docs alineado con el SemVer por documento del D8; ecosistema de plugins; escalable a un sitio público con marca. Contras: introduce una toolchain **Node/npm ajena al stack Python** actual (más superficie de mantenimiento para un equipo de 1 humano — tensión con ADR-0004 regla 13); MDX puede acoplar el contenido a componentes; mayor complejidad de build.

## D. Enterprise — Docusaurus + versionado + búsqueda + CI

Docusaurus con: versionado de docs activado (una versión publicada por versión mayor de la Bible), búsqueda (Algolia hospedada o índice local offline) y un pipeline CI que construye y despliega en cada merge, con verificación automática de enlaces rotos y de la Definition of Done (1.4 §8).

Pros: publicación reproducible y **automatizada**; docs versionadas navegables; búsqueda de calidad; control de calidad continuo. Es la lectura literal más fiel del D3 (*"prioriza calidad y escalabilidad"*). Contras: **máxima** complejidad operativa y de mantenimiento; la búsqueda Algolia hospedada implica un servicio externo (tensión con ADR-0001, salvo usar búsqueda local); CI/CD exige infraestructura y disciplina; **sobredimensionado** para la fase documental actual (un solo autor humano) — choca con la salvaguarda anti-sobre-ingeniería (ADR-0004 regla 13) sin un caso de evolución hoy documentado.

### Recomendación del proponente (Fable, CTO)†

**Nivel Balanceado — MkDocs Material** como elección inmediata, conservando `herramientas/publicar.py` como utilidad complementaria para el PDF del D12 mientras se migra. Argumento:

1. **Honra el D8 al pie de la letra:** es uno de los dos generadores que el CEO nombró.
2. **Permanece en el stack Python** ya presente, con curva de adopción mínima y mantenimiento sostenible por el equipo real (ADR-0004 regla 13).
3. **Resuelve las tres carencias del publicador actual** (alternativa A): búsqueda, navegación jerárquica y render de Mermaid — mejorando la legibilidad humana e IA que exige el D13.
4. **Es 100% local y estático:** refuerza el ADR-0001 (sin servicios externos ni telemetría de terceros).
5. **Deja un camino de evolución documentado** hacia Profesional/Enterprise (Docusaurus + versionado + búsqueda + CI) para cuando se materialice un disparador concreto —publicación pública de la Bible, varias versiones mayores vivas en paralelo, o colaboradores externos—, respetando la Evolución Permanente (ADR-0004) sin pagar hoy su costo.

Sobre el D3: aunque el acta no fija techo presupuestal y prioriza calidad/escalabilidad, la salvaguarda anti-sobre-ingeniería que el CEO aprobó en el gate D2 (ADR-0004 regla 13) fija la **sostenibilidad** como criterio de desempate; por eso el proponente recomienda B y no D. La decisión final es del CEO: *"el CEO decidirá cuál implementar"* (D3).

## Decisión

**Aceptada por el CEO (2026-07-18): Nivel Balanceado — MkDocs Material** como estándar documental del proyecto. Criterios del CEO: local-first; integración nativa con Python; excelente navegación y búsqueda; soporte para Mermaid; bajo costo de mantenimiento; posibilidad de migración futura sin afectar la documentación. Se conserva `herramientas/publicar.py` como herramienta transitoria del D12 (generación de PDF) hasta que MkDocs cubra ese punto. El contenido permanece en Markdown plano, sin lock-in del generador (coherente con D19: migración futura sin rediseño).

## Consecuencias

**Positivas (nivel recomendado B):** sitio con búsqueda, navegación y diagramas renderizados; se cumple el D8 con mínima deuda técnica; el contenido sigue siendo Markdown plano versionable (sin lock-in del generador). **Negativas:** una dependencia de build nueva (MkDocs + plugins) y una migración inicial; el PDF requiere ruta propia hasta consolidarse. **Riesgos aceptados:** si en el futuro se necesita versionado de docs o publicación pública automatizada, habrá que evolucionar a C/D mediante un ADR nuevo (previsto como camino documentado, no como límite artificial). Si el CEO elige D, se asume el mayor costo de mantenimiento y, de usar Algolia hospedada, se debe preferir búsqueda local para no tensar el ADR-0001.

## Impacto (acta D11 — obligatorio)

Evaluación del **nivel recomendado (B, MkDocs Material)**; entre paréntesis, la variación relevante del nivel Enterprise (D).

| Dimensión | Impacto |
|---|---|
| Seguridad | Positivo: sitio estático local, sin backend ni superficie de ataque nueva (D: el CI y Algolia hospedada añaden superficie e integraciones externas a asegurar). |
| Privacidad | Positivo: 100% local, sin telemetría de terceros; refuerza ADR-0001 (D: Algolia hospedada enviaría el índice a un tercero — usar búsqueda local para mantener la postura). |
| Modularidad | Neutro/positivo: el contenido sigue en Markdown plano, desacoplado del generador; cambiar de generador no reescribe los documentos. |
| Costo | Muy bajo: herramienta libre, sin infraestructura; esfuerzo de migración acotado (D: costo alto de montaje y operación de CI + búsqueda). |
| Experiencia de usuario | Positivo alto: búsqueda, navegación jerárquica y Mermaid renderizado mejoran la legibilidad humana/IA (D13); (D añade docs versionadas y despliegue automático). |
| Mantenimiento futuro | Positivo controlado: bajo mantenimiento, sostenible por el equipo real (ADR-0004 regla 13); (D aumenta el mantenimiento con la toolchain Node + CI). |

## Trazabilidad

Capítulos a actualizar: **1.4** (este capítulo referencia el ADR como propuesta y nace citándolo; al aceptarse, 1.4 registrará la plataforma elegida en su "Decisiones aplicables"). Herramienta afectada: `herramientas/publicar.py` (es la totalidad de la alternativa A; en B/C/D pasa a utilidad complementaria de PDF o se retira según lo decida el CEO). Módulos de código afectados (futuro): "—" mientras no exista código de producto.

---